KEY TERM

Set: a collection of data items that lacks any structure; contains no duplicates and has a number of defined operations that can be performed on it

The most useful property of a set is the fact that duplicate values are not allowed. A list or a one-dimensional array might be created but has to be checked to remove duplicate values. A simple way of removing duplicate values would be to convert the structure to a set and then convert the set back to the original structure.

A slightly different example would be if students were allocated to groups for studying a particular subject. For each subject, the students' names would be entered into a data structure defined for that subject. Set data types could then, for example, find out which students were studying both computer science and physics. The students studying both subjects would be found by applying the 'intersection' operation on the two individual sets.

16.02 File organisation

In everyday computer usage, a wide variety of file types is encountered. Examples are graphic files, word-processing files, spreadsheet files and so on. Whatever the file type, the content is stored using a specific binary code that allows the file to be used as intended.

For the specific task of storing data to be used for input to a computer program or for output from a computer program, there are only two defined file types. A file is either a text file or a **binary file**.

A text file, as discussed in Chapter 13 (Section 13.06) contains data stored according to a character code of the type described in Chapter 1 (Section 1.04). It is possible, by using a text editor, to create a text file to be used as input to a program.

A program may create a binary file as output with the intention of subsequently using it for input. A binary file stores data in its internal representation, for example an integer value might be stored in two bytes in two's complement representation.

The organisation of a binary file is based on the concept of a **record**. A file contains records and each record contains fields. Each field consists of a value. For a text file the number of data items per line must be known and the number of characters per item must be known. If these are not known then item separator characters must be used. The file has repeating lines which are defined by an end-of-line character or characters.

For a binary file the number of fields per record must be known. If any of the fields represent a string, the length of the string must be known. For any other field the internal representation will define the number of bytes required to store the field value. There is no need for field separator characters or for an end-of-record character.

KEY TERMS

Binary file: a file designed for storing data to be used by a computer program

Record: a collection of fields containing data values

Discussion Point:

A record is a user-defined data type. It is also a component of a file. Can there be or should there be any relationship between these two concepts?

Serial files

A serial file contains records that have not been organised in any defined order. A typical use of a serial file would be for a bank to record transactions involving customer accounts. A program would be running. Each time there was a withdrawal or a deposit the program would receive the details as data input and would record the data in a transaction file. In a serial file each new record is simply appended to the file so that the only ordering in the file is the time order of data entry.

Sequential files

A sequential file has records that are ordered. In the bank example, a sequential file could be used as a master file for an individual customer account. At regular periods of time, the transaction file would be read, and all affected customer account master files would be updated. In order to allow a sequential file to be ordered, there has to be a key field for which the values are unique and sequential but not necessarily consecutive. When a new record is to be added to a sequential file it would be possible to simply append the record, with the intention of sorting the file later. A more likely approach is for the file to be read sequentially and each record written to a new file. This is continued until the appropriate position for the new record is reached. The new record is then written to the new file before the remaining records in the old file are copied in.

Extension Question 16.01

Can you think of reasons why you might want to use binary files with variable-length records? How would you make sure a binary file with variable-length records would be read correctly?

Direct-access files

Direct-access files are sometimes referred to as 'random-access' files but, as with random-access memory, the randomness is only that the access can be to any record in the file without sequential reading of the file. Direct access can be achieved with a sequential file. A separate index file is created which has two fields per record. The first field has the key field value and the second field has a value for the position of this key field value in the main file.

The alternative is to use a hashing algorithm when a record is entered into the direct-access file.

One simple hashing algorithm is applicable if there is a numeric key field in each record. The algorithm chooses a suitable number and divides this number by the value in the key field. The remainder from this division then identifies the address in the file for storage of that record. The suitable number works best if it is a prime number of a similar size to the expected size of the file.

For simplicity this can be illustrated for 4-digit values in the key field where 1000 is used for the dividing number. The following represent three calculations:

0045/1000 gives remainder 45 for the address in the file

2005/1000 gives remainder 5 for the address in the file

3005/1000 gives remainder 5 for the address in the file

There are two facts apparent from these calculations. The first fact is that the addresses calculated do not have any order depending on the value in the key field. The second fact is that different key field values can produce the same remainder and therefore the same address in the file.

If the records do not have a suitable field with numeric digits, an alternative is to choose a field with some alphabetic characters. The ASCII code for each character can be looked up and the values then added. The sum is then used in the same way as described above, to calculate an address as the remainder from an integer division.

When the same address is calculated for different field values, it is usually referred to as a collision (the addresses are sometimes called synonyms). The best choice for a hashing algorithm is one that spreads the addresses most evenly and minimises the number of collisions. However, collisions cannot be avoided altogether so there has to be a defined method for dealing with collisons. There are a number of options, including the following:

- use a sequential search to look for a vacant address following the calculated one
- keep a number of overflow addresses at the end of the file
- have a linked list accessible from each address.

Question 16.01

Imagine the possible numeric values for a key field in a direct-access file are in the range of 1 to 30 but you want the file to have fewer than 30 file addresses.

You decide to test two examples of a modular division hashing algorithm. The first test uses 10 as the number for division, the second test uses 11.

a What are the two sets of addresses generated as remainders from the division for the key values 0 to 39 using 10 and 11?

b State one difference between the two sets of addresses.

c Is there any significant difference between the two sets of addresses?

d 11 is a prime number. Prime numbers are stated to give a better spread of use of the addresses in a file. Do you know when this is more likely to be true?

File access

Once a file organisation has been chosen and the data has been entered into a file, you need to consider how this data is to be accessed. For a serial file, the normal usage is to read the whole file record by record. If there was a need to search for a particular value in one of the fields, the only option would be to read the records from the beginning until the target record was found. If the data is stored in a sequential file and a particular value is needed, searching may have to be done in the same way. However, if the key field value is known for the record containing the wanted data, the process is faster because only key field values need to be read. For a direct-access file, the value in the key field is submitted to the hashing algorithm. The value is the same value that was used when entering the data originally and will provide the same value for the position in the file that was provided when the algorithm was used at the time of data input. This eliminates the need to read records from the beginning of the file. However, because of the collision problem some serial searching might be needed after the initial jump to the hashed position.

File access might also be needed to delete or edit data. For a sequential file the same method is used as when a new record was added. Records are copied from the old file to a new file until the record that needs to be deleted or edited is reached. Following deletion or editing all remaining records are copied to the new file.

For a direct-access file there is no need to create a new file. If a record needs editing it can be accessed directly and edited without disturbing any other content. However, if a record

is to be deleted it is necessary to have a flag set in the record. Then, in a subsequent reading process, that record is skipped over.

Choice of file organisation

Serial file organisation is well suited to batch processing or for backing up data on magnetic tape. A direct access file is used if rapid access to an individual record in a large file is required. An example would be on a system with many users. In this case, the file that is used to check passwords when users log in should be direct-access. A sequential file is suitable for applications when multiple records are required from one search of the file. An example could be a family history file where a search could be used for all records with a particular family name.

At this point it is worth mentioning the difference between key fields in a file and primary keys in a database table. In the database table the primary key values must all be unique. This is not a requirement for key fields in any type of file. It may be sensible in certain applications to ensure key fields have unique values, but it is not mandatory.

16.03 Real numbers

A real number is one with a fractional part. When we write down a value for a real number in the denary system we have a choice. We can use a simple representation, or we can use an exponential notation (sometimes referred to as scientific notation). For example, the number 25.3 might be written as:

$$.253 \times 10^{2} \text{ or } 2.53 \times 10^{1} \text{ or } 25.3 \times 10^{0} \text{ or } 253 \times 10^{-1}$$

For this number, the simple expression is best. But if a number is very large or very small the exponential notation is the only sensible choice.

Floating-point and fixed-point representations

A binary code must be used for storing a real number in a computer system. One possibility is to use a fixed-point representation. In fixed-point representation, an overall number of bits is chosen with a defined number of bits for the whole number part and the remainder for the fractional part. The alternative is a **floating-point representation**. The format for a floating-point number can be generalised as:

$$\pm M \times R^{E}$$

In floating-point representation a defined number of bits are used for what is called the significand or mantissa, ±M. The remaining bits are used for the exponent or exrad, E. The radix, R is not stored in the representation; R has an implied value of 2.



KEY TERM

Floating-point representation: a representation of real numbers that stores a value for the mantissa and a value for the exponent

A simple example can be used to illustrate the differences between the two representations. Let's consider that a real number is to be stored in eight bits.